AWS DEVELOPER LAB · DISCOVERY REPORT

Safe Order System Delivery for Onward Logistics

Delivering Order System Updates Safely

Prepared by Patrick Ndaziramiye
Date November 2026
Region eu-west-1

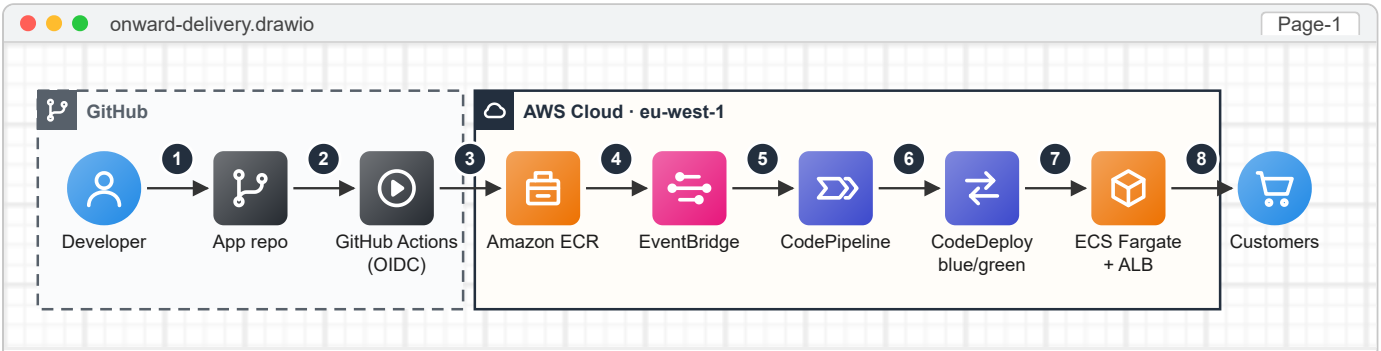


Figure 1 · From merge to customers: the eight automated steps of one release

Goal
Every approved change reaches customers automatically, safely, with zero downtime and easy rollback.

Approach
ECS Fargate behind one ALB, released by CodeDeploy blue/green with canary traffic and alarm rollback.

Security
No stored keys (GitHub OIDC), one public entry point, no internet path for the app.

Cost
About $94/month while running; teardown makes an idle environment cost nothing.

Contents

1. Part A.1 · How teams deliver changes today
2. Part A.2 · What happens without a solution
3. Part B.1 · Chosen approach and architecture
4. Part B.2 · Services, and why each one
5. Part B.3 · Who benefits
6. Sources

Icon colours

Compute & containers · Networking · Security & identity · Developer tools · Integration & management · Storage · GitHub

A.1 How teams deliver changes: common solutions

Release practices range from hands-on manual deploys to fully automated, progressive releases. Moving right in Figure 2 lowers the chance that a bad change reaches customers, and the time it takes to undo one.

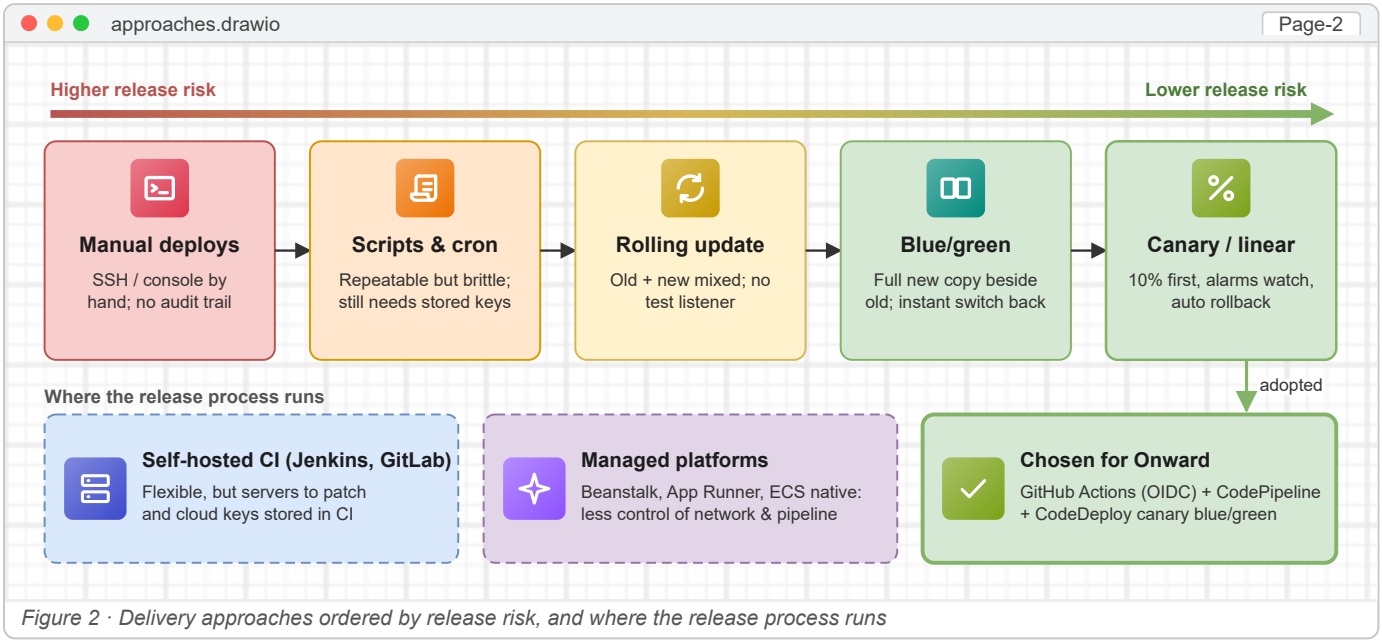


Figure 2 · Delivery approaches ordered by release risk, and where the release process runs

Approach	How it works	Strength	Weakness for Onward
Manual deploys	Engineer copies files and restarts the app	No tooling needed	Slow, error-prone, standing credentials, no audit trail
Scripts / cron	Shell scripts run the same steps	Repeatable	Brittle, still needs keys, no traffic control
Rolling update	Tasks replaced a few at a time	Built into ECS, no extra cost	Old and new serve together; rollback is another rollout
Blue/green	Full new copy starts beside the old one; traffic switches	Instant switch back; test before going live	Briefly runs two copies
Canary / linear	A small share of traffic goes to the new version first	Limits how many users a bad release reaches	Needs health signals (alarms)
Jenkins / GitLab CI	Self-hosted or SaaS CI/CD servers	Very flexible	Servers to patch; often long-lived cloud keys
Beanstalk, App Runner, ECS native	Managed release features	Less to build	Less control over the private network and pipeline the brief requires

Sources: [1] Well-Architected OPS06 · [2] Blue/Green Deployments on AWS · [3] ECS blue/green with CodeDeploy · [4] CodeDeploy deployment configurations.

A.2 What happens without a solution

Without an automated, reversible release process, each change is a risk to the order system. Figure 3 shows the six consequences, and each one costs Onward either orders, security, or engineering time.

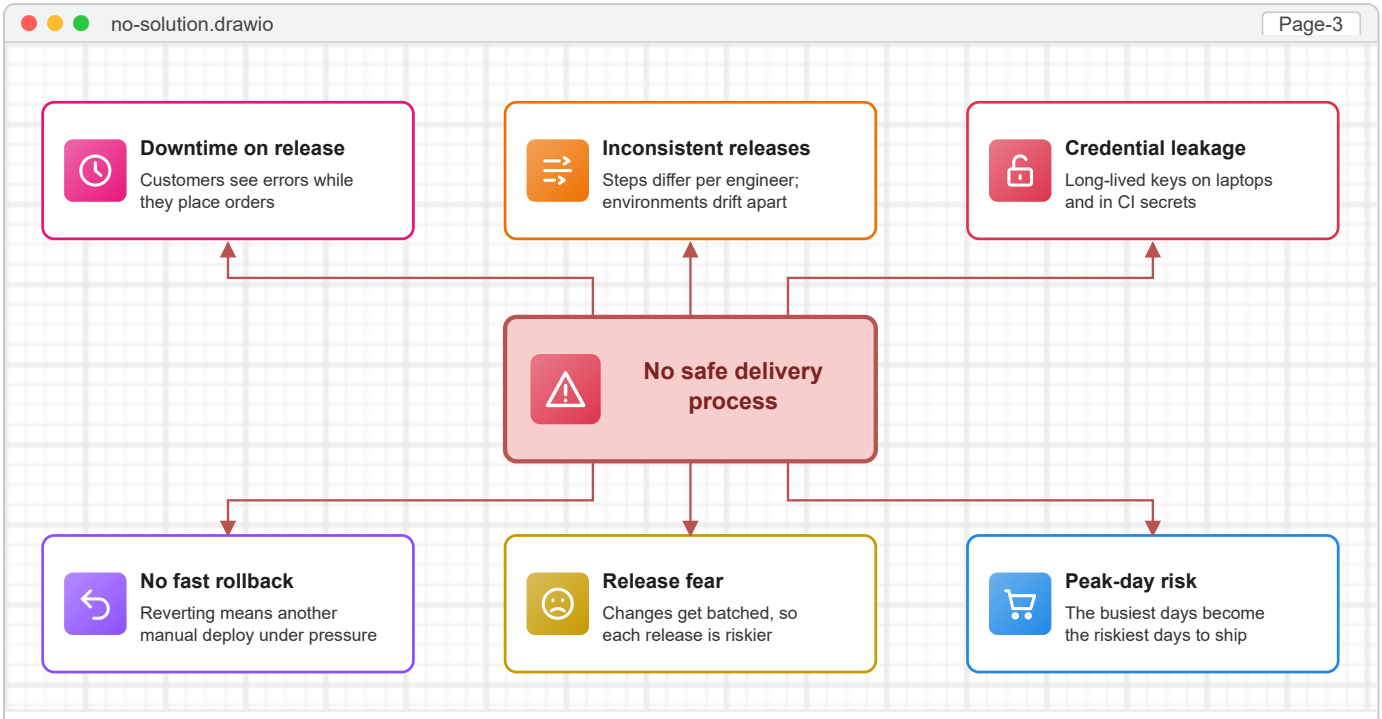


Figure 3 · Consequences of shipping without a safe delivery process

Business impact

- **Lost orders.** Every minute of downtime during a release is revenue that does not come back.
- **Breach exposure.** Long-lived access keys are a leading cause of cloud security incidents [8].
- **Slow change.** Fear of releases leads to large, infrequent batches, which make every release riskier [1].
- **Hard to rebuild.** Hand-made environments drift, so problems cannot be reproduced or recovered from quickly.

Sources: [1] Well-Architected OPS06 · [5] GitHub OIDC for AWS · [8] Well-Architected SEC02 temporary credentials.

B.1 Chosen approach and architecture

ECS on Fargate in private subnets behind a single ALB. GitHub Actions (OIDC) pushes the image to ECR. EventBridge starts CodePipeline, and CodeDeploy releases it blue/green with canary traffic and alarm-based automatic rollback. All infrastructure is CloudFormation deployed by GitSync, and AWS services are reached through VPC endpoints, not a NAT gateway.

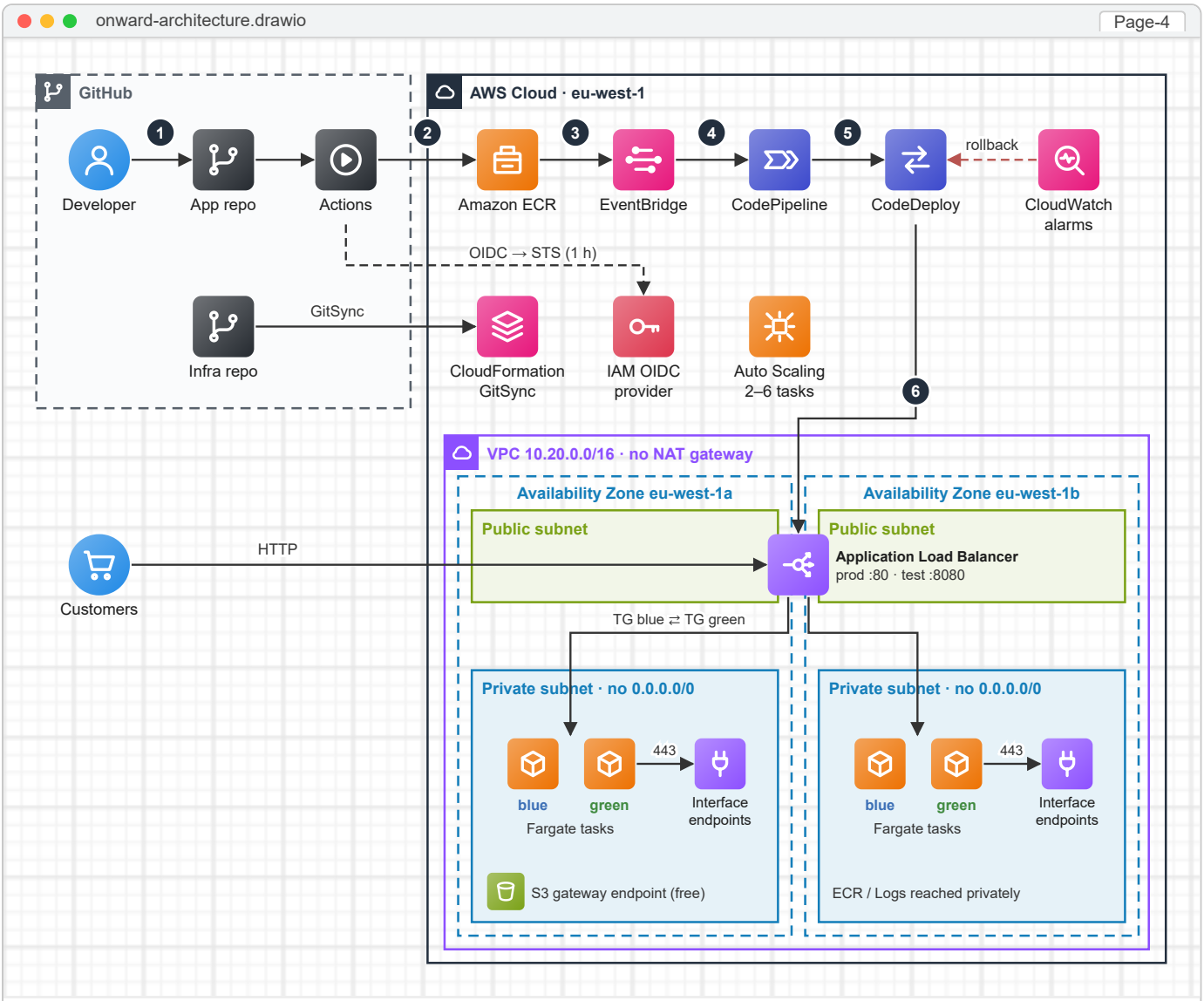


Figure 4 · Target architecture and the numbered flow of one change

1. **Merge** to `main` in the app repo. This is the only manual step.
2. **GitHub Actions** tests, assumes an IAM role through OIDC (1-hour credentials, no stored keys), and pushes `:<sha>` and `:latest` to **ECR**.
3. ECR emits an **Image Action PUSH** event.
4. An **EventBridge** rule starts **CodePipeline**. There is no polling.
5. The pipeline registers a new task definition and calls **CodeDeploy**.
6. CodeDeploy starts the green tasks in both AZs, shifts **10% of traffic for 5 minutes, then 100%**, and **rolls back automatically** if an alarm fires.

B.2 Services, and why each one

Each service in the solution is there because a stakeholder asked for what it provides. Figure 5 traces every stakeholder need to the services that meet it.

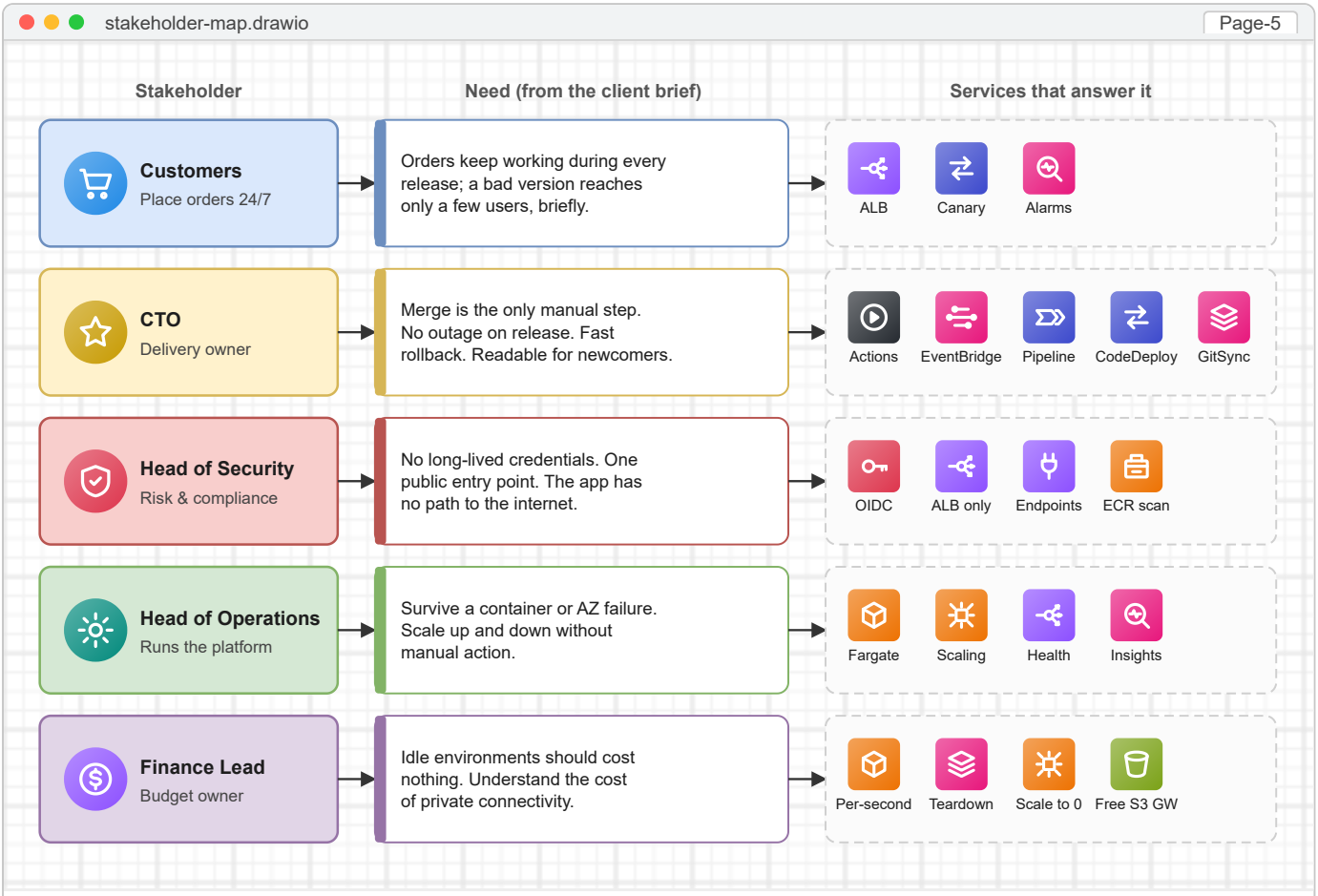


Figure 5 · Stakeholder needs mapped to AWS services

Service	Why it is used	Serves
Application Load Balancer	Single public entry point, health checks, prod + test listeners for blue/green	Security, CTO
Amazon ECS on Fargate	No servers to patch; tasks spread over two AZs	Operations
Application Auto Scaling	CPU/memory target tracking, 2–6 tasks; scale to zero when idle	Operations, Finance
Amazon ECR	Private registry, scan on push, immutable SHA tags for rollback	Security, CTO
VPC endpoints	ECR, Logs and S3 reached without any internet route	Security
IAM + GitHub OIDC	Temporary STS credentials; trust limited to one repo and branch	Security
Amazon EventBridge	Image push event starts the release, with no polling	CTO
CodePipeline + CodeDeploy	Managed blue/green, canary shift, automatic rollback	CTO, Customers
Amazon CloudWatch	5XX and unhealthy-host alarms stop bad releases	CTO, Operations
CloudFormation GitSync	Reviewed, repeatable infrastructure; rebuild or tear down from Git	CTO, Finance

B.3 Who benefits

The same release, before and after. The manual chain of risky steps becomes an automated chain where the only human action is approving a merge.

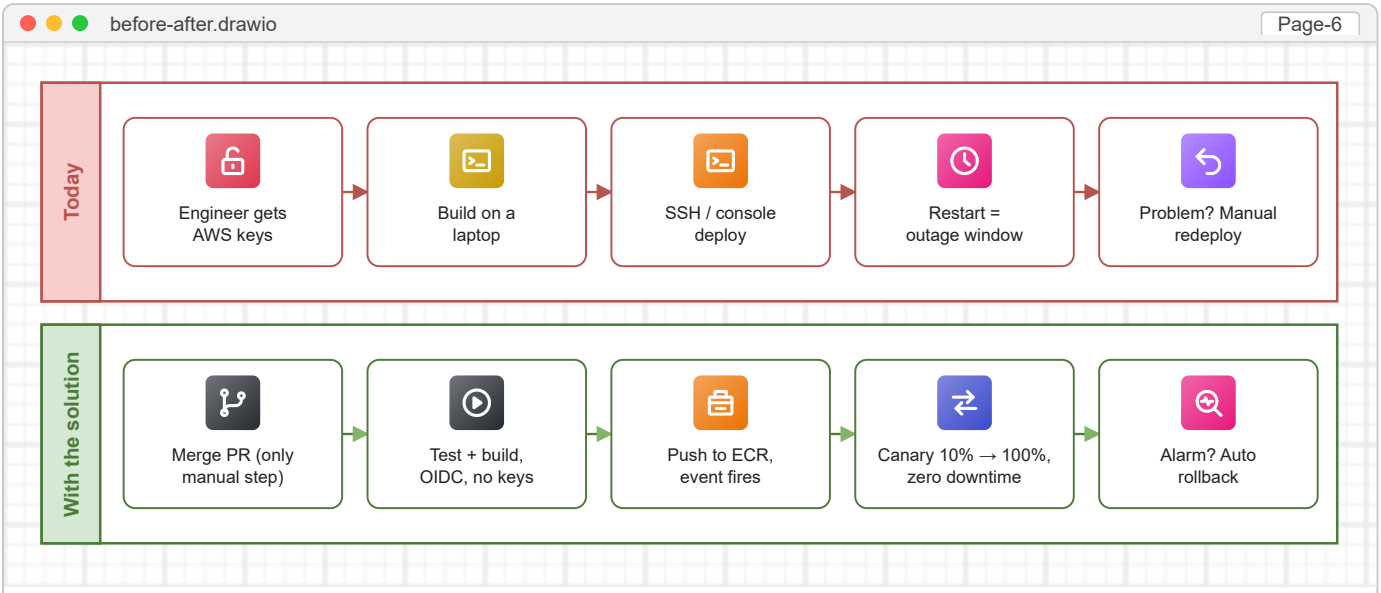


Figure 6 · Release process today vs with the solution

Customers
No outage during releases. A bad release reaches at most 10% of traffic, for minutes, before rollback.

CTO
Merging is the whole release process. Releases become frequent, small, and routine.

Head of Security
No keys to leak, one public door, and an application that cannot reach the internet.

Head of Operations
Self-healing tasks across two AZs, automatic scaling, and alarms watching every release.

Finance Lead
Per-second Fargate billing, scale-to-zero or teardown when idle, and a clear endpoint cost.

Engineers
Two small, documented repos, and the same environment rebuilt from code every time.

Cost in one line

About $94/month while running (eu-west-1): ~$48 interface endpoints, ~$24 ALB, ~$18 Fargate, plus small amounts for logs and alarms. Endpoints cost about $13 more than one NAT gateway, but NAT would give the app an internet path, which Security rules out. Teardown brings the idle cost to zero.

Sources

[1] AWS Well-Architected Framework, Operational Excellence: OPS06 Mitigate deployment risks
https://docs.aws.amazon.com/wellarchitected/latest/operational-excellence-pillar/

[2] AWS Whitepaper: Blue/Green Deployments on AWS
https://docs.aws.amazon.com/whitepapers/latest/blue-green-deployments/

[3] Amazon ECS Developer Guide: Blue/green deployment with CodeDeploy
https://docs.aws.amazon.com/AmazonECS/latest/developerguide/deployment-type-bluegreen.html

[4] AWS CodeDeploy User Guide: Deployment configurations on an ECS compute platform
https://docs.aws.amazon.com/codedeploy/latest/userguide/deployment-configurations.html

[5] GitHub Docs: Configuring OpenID Connect in Amazon Web Services
https://docs.github.com/en/actions/security-for-github-actions/security-hardening-your-deployments/configuring-openid-connect-in-amazon-web-services

[6] Amazon ECR User Guide: Interface VPC endpoints (AWS PrivateLink)
https://docs.aws.amazon.com/AmazonECR/latest/userguide/vpc-endpoints.html

[7] AWS CloudFormation User Guide: Git sync
https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/git-sync.html

[8] AWS Well-Architected Framework, Security: SEC02 Use temporary credentials
https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/

[9] AWS Pricing Calculator (prices used in the cost estimate)
https://calculator.aws/

Assumptions

Stakeholder needs are summarised from the client brief. They are not verbatim quotes.
Prices are approximate eu-west-1 on-demand list prices. Confirm them in the AWS Pricing Calculator.
Naming and tags follow the pattern `onward-orders-lab-*` with Project, Environment, Owner, CostCenter, ManagedBy and Repository tags.

Full implementation: **onward-orders-infra** (CloudFormation + GitSync) and **onward-orders-app** (code, Dockerfile, workflow, appspec, task definition). Detailed costs are in `docs/cost-analysis.md`.